Lecture 23 — Segment Trees

> Year Curriculum · Prerequisites: Week 3 (complexity), Week 7 (prefix sums), Week 9 (bit manipulation), Week 22 (Euler tour) · Estimated time: 2.5 hours

1. Overview

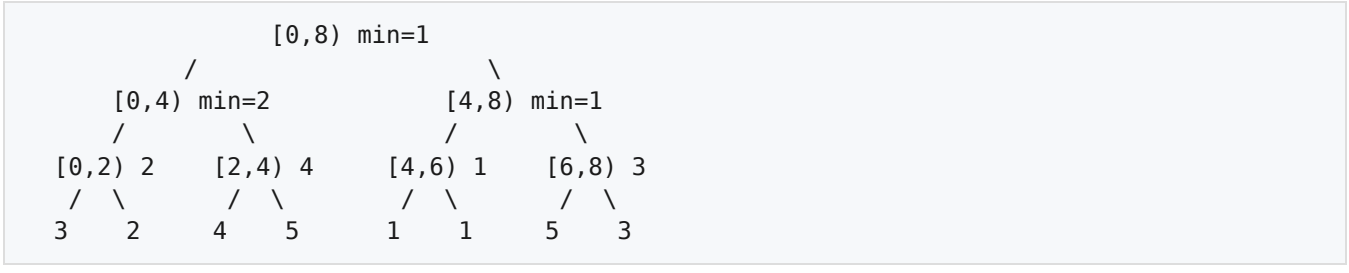
Prefix sums (Week 7) answer range-sum queries in O(1), but a single change to the array forces an O(n) rebuild. Real problems interleave **updates** and **queries**: change `a[k]`, ask for the minimum of `a[l..r]`, change another element, ask again. The **segment tree** handles both operations in O(log n) and is the most versatile data structure in competitive programming. It supports any associative operation (sum, min, max, gcd, xor, matrix product, "max subarray sum"...), can be "walked" to find the first or k-th element satisfying a property, and (next-next week) can be extended with lazy propagation to support range updates.

This lecture covers:

- the structure and why it gives O(log n) operations;
- a recursive implementation (flexible) and a bottom-up iterative one (short and fast);
- designing custom node types;
- descending ("walking") the tree;
- segment trees indexed by value.

2. Structure

Take an array `a[0..n-1]`. The root of the tree represents the whole range `[0, n-1]`. A node representing `[l, r]` with `l < r` has two children representing `[l, m]` and `[m+1, r]` where `m = (l + r) / 2`. Leaves represent single positions. Every node stores the combined value (say, the minimum) of its range.



Properties:

- The height is ⌈log2 n⌉, and there are fewer than 2n nodes.

- Each position belongs to exactly one node per level, so to exactly ⌈log2 n⌉ + 1 nodes.
- Storing in an array with root at index 1 and children of `x` at `2x` and `2x+1` needs up to `4n` cells when n is not a power of two (the tree is not complete, so some indices are skipped). Using `4n` is the standard safe choice.

2.1 Point update

To set `a[pos] = v`, walk from the root to the leaf of `pos`, set the leaf, and recompute each ancestor on the way back: `t[x] = combine(t[2x], t[2x+1])`. One root-to-leaf path: O(log n).

2.2 Range query

To query `[ql, qr]` at node `x` with range `[l, r]`:

- **Disjoint** (`qr < l` or `r < ql`): return the identity of the operation (`+∞` for min, 0 for sum).
- **Fully covered** (`ql ≤ l` and `r ≤ qr`): return `t[x]`.
- **Partial**: recurse into both children and combine.

Why O(log n). Call a node *partial* if it overlaps the query without being covered. A partial node must contain one of the query borders, i.e., contain both `ql−1` and `ql`, or both `qr` and `qr+1`. Each border is inside exactly one node per level, so there are at most 2 partial nodes per level. Only partial nodes recurse, each to 2 children, so at most 4 nodes per level are visited and the total is at most 4⌈log2 n⌉. The covered nodes form the canonical decomposition of `[ql, qr]` into at most 2⌈log2 n⌉ dyadic pieces.

Example with n = 8, query `[1, 6]`: the covered nodes are `[1]`, `[2,3]`, `[4,5]`, `[6]`.

2.3 Which operations work?

The combine must be **associative**: `(a ⊕ b) ⊕ c = a ⊕ (b ⊕ c)`, because the tree groups elements in an arbitrary but order-preserving way. It need not be commutative in the recursive version, since children are always combined left-then-right. It needs an identity element (or you can handle "empty" results specially). Sum, min, max, gcd, xor, bitwise and/or, matrix multiplication, and "composition of linear functions" all qualify. Note that "number of distinct values" does not have a small mergeable summary, so it is not a plain segment-tree query (see CSES 1734, solved offline).

3. The bottom-up (iterative) segment tree

If the operation is commutative, an extremely compact version exists. Store leaves at `t[n .. 2n−1]` and internal nodes at `t[1 .. n−1]` with `t[i] = combine(t[2i], t[2i+1])`. This works for any n, not just powers of two (some internal nodes then cover "wrapped" sets of leaves, but every query is still decomposed correctly).

Update: set the leaf `t[p+n]`, then move to the parent repeatedly, recomputing `t[p>>1] = combine(t[p], t[p^1])` (`p^1` is the sibling).

Query on the half-open range `[l, r)`: shift both ends by `n` and climb. If `l` is odd, it is a right child whose parent extends to the left of the range, so take `t[l]` alone and step `l` to the right. If `r` is odd, then `r−1` is a left child completely inside, so take `t[r−1]`. Then halve both. The loop stops when `l ≥ r`.

For a **non-commutative** combine (e.g., a string hash or a matrix product) use two accumulators: pieces taken from the left end go on the right of `resL` (`resL = resL ⊕ t[l++]`), pieces from the right end go on the left of `resR` (`resR = t[--r] ⊕ resR`), and the answer is `resL ⊕ resR`. With non-power-of-two n, the order of leaves inside some internal nodes is broken, so for non-commutative operations pad n up to a power of two.

The iterative version is usually 2–3 times faster than the recursive one and uses exactly `2n` memory.

4. Custom nodes

The art of segment trees is choosing what a node stores. Ask: *what does a parent need to know that it can compute from its two children alone?*

Maximum subarray sum (CSES 1190). The best subarray of a parent is either entirely in the left child, entirely in the right child, or crosses the middle. A crossing subarray is a suffix of the left part plus a prefix of the right part. So each node stores four values: `sum`, best prefix `pref`, best suffix `suf`, and best overall `best`:

```
sum  = L.sum + R.sum
pref = max(L.pref, L.sum + R.pref)
suf  = max(R.suf, R.sum + L.suf)
best = max(L.best, R.best, L.suf + R.pref)
```

Minimum and the number of times it occurs. Node = (min, count). When merging, if the minima are equal add the counts, otherwise take the smaller one with its count. Codeforces 474F *Ant colony* uses gcd together with this: an ant survives iff its strength equals the gcd of the range, i.e. it equals the minimum and the minimum equals the gcd.

Bracket sequences (Codeforces 380C). Node = (matched pairs, unmatched '(' , unmatched ')'). Merging matches `min(L.open, R.close)` new pairs.

5. Walking (descending) the tree

Many questions have the form "find the first position where something happens". A binary search over the answer with a range query inside costs O(log² n). Walking the tree does it in O(log n) by using the stored aggregates to decide which child to enter.

First position with value ≥ x (CSES 1143 *Hotel Queries*). Store the maximum. At a node, if its max is `< x` there is no answer inside. Otherwise go to the left child if its max is `≥ x` (leftmost preferred), else to the right child. One root-to-leaf path: O(log n).

k-th one. On a 0/1 array with a sum tree: at a node, if the left child contains at least k ones go left, else go right with `k − left`. This finds the k-th remaining element (CSES 1749 *List Removals*) or, on a tree indexed by value, the k-th smallest element of a multiset.

With a left bound. "First index `≥ L` with `a[i] ≥ x`" is answered by a range-query-style traversal of `[L, n−1]` that, upon reaching a fully covered node with max `≥ x`, descends inside it to the leftmost qualifying leaf and stops. Covered nodes are visited left to right, so the first one that qualifies contains the answer; total work is still O(log n).

6. Segment trees over values

Instead of positions, index the tree by (compressed) values: leaf `v` stores how many elements currently equal `v`. Then "how many elements are in `[lo, hi]`" is a range sum and "k-th smallest" is a k-th-one walk. CSES 1144 *Salary Queries* (change one salary, count salaries in a value range) is solved by compressing all values that appear in the input and queries, then using a sum tree over the compressed indices. Values up to 10^9 must be compressed first, otherwise the tree is too large (or use a dynamic/sparse segment tree — a later topic).

7. Complexity summary

Operation	Time
Build	O(n)
Point update	O(log n)
Range query	O(log n)
Walk / descent	O(log n)
Memory	`2n` iterative, `4n` recursive

With n = q = 2·10^5, a query visits at most about 72 nodes, so the total is a few million operations.

8. Reference implementations

8.1 Iterative segment tree (CSES 1649 *Dynamic Range Minimum Queries*)

Verified on the CSES sample and against a brute force on random arrays of sizes 1–13 (non-powers of two included).

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1649 - Dynamic Range Minimum Queries: iterative (bottom-up) segment tree.
struct SegTree {
    int n;
    vector<int> t;
    SegTree(const vector<int>& a) : n(a.size()), t(2 * a.size()) {
        for (int i = 0; i < n; i++) t[n + i] = a[i];
        for (int i = n - 1; i >= 1; i--) t[i] = min(t[2 * i], t[2 * i + 1]);
    }
    void update(int p, int value) {          // a[p] = value, 0-indexed
        for (t[p += n] = value; p > 1; p >>= 1) t[p >> 1] = min(t[p], t[p ^ 1]);
    }
    int query(int l, int r) const {          // min of a[l..r), 0-indexed, half-open
        int res = INT_MAX;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res = min(res, t[l++]);
            if (r & 1) res = min(res, t[--r]);
        }
        return res;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int& x : a) cin >> x;
    SegTree st(a);
    while (q--) {
        int type, x, y;
        cin >> type >> x >> y;
        if (type == 1) st.update(x - 1, y);
        else cout << st.query(x - 1, y) << '\n';
    }
}
```

8.2 Recursive segment tree with a walk (CSES 1143 *Hotel Queries*)

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1143 - Hotel Queries: recursive max segment tree + "walking" descent.
int n;
vector<int> h, mx;

void build(int node, int l, int r) {
    if (l == r) { mx[node] = h[l]; return; }
    int m = (l + r) / 2;
    build(2 * node, l, m);
    build(2 * node + 1, m + 1, r);
    mx[node] = max(mx[2 * node], mx[2 * node + 1]);
}

void update(int node, int l, int r, int pos, int value) {
    if (l == r) { mx[node] = value; return; }
    int m = (l + r) / 2;
    if (pos <= m) update(2 * node, l, m, pos, value);
    else update(2 * node + 1, m + 1, r, pos, value);
    mx[node] = max(mx[2 * node], mx[2 * node + 1]);
}

// Leftmost index i with h[i] >= need, or -1. O(log n): we only ever go down.
int firstAtLeast(int node, int l, int r, int need) {
    if (mx[node] < need) return -1;
    if (l == r) return l;
    int m = (l + r) / 2;
    if (mx[2 * node] >= need) return firstAtLeast(2 * node, l, m, need);
    return firstAtLeast(2 * node + 1, m + 1, r, need);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> n >> m;
    h.resize(n);
    mx.assign(4 * n, 0);
    for (int& x : h) cin >> x;
    build(1, 0, n - 1);
    for (int i = 0; i < m; i++) {
        int r;
        cin >> r;
        int idx = firstAtLeast(1, 0, n - 1, r);
        if (idx == -1) {
            cout << 0 << ' ';
        } else {
            cout << idx + 1 << ' ';
            h[idx] -= r;
            update(1, 0, n - 1, idx, h[idx]);
        }
```

```
    }
    cout << '\n';
}
```

8.3 Custom node: maximum subarray sum (CSES 1190 *Subarray Sum Queries*)

Here the size is padded to a power of two (padding leaves are zeros, which do not change the answer because the empty subarray is allowed).

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1190 - Subarray Sum Queries: segment tree with a 4-field node.
// After each point update, print the maximum subarray sum (empty allowed).
struct Node {
    long long sum, pref, suf, best;
};
Node makeLeaf(long long v) {
    long long p = max(0LL, v);
    return {v, p, p, p};
}
Node combine(const Node& L, const Node& R) {
    return {L.sum + R.sum,
            max(L.pref, L.sum + R.pref),
            max(R.suf, R.sum + L.suf),
            max({L.best, R.best, L.suf + R.pref})};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int sz = 1;
    while (sz < n) sz <<= 1;                // power of two keeps left-to-right order
    vector<Node> t(2 * sz, makeLeaf(0));
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        t[sz + i] = makeLeaf(x);
    }
    for (int i = sz - 1; i >= 1; i--) t[i] = combine(t[2 * i], t[2 * i + 1]);
    while (m--) {
        int k; long long x;
        cin >> k >> x;
        int p = sz + k - 1;
        t[p] = makeLeaf(x);
        for (p >>= 1; p >= 1; p >>= 1) t[p] = combine(t[2 * p], t[2 * p + 1]);
        cout << t[1].best << '\n';
```

```
    }
}
```

9. Common mistakes

- **Too little memory.** The recursive tree with root 1 needs `4n` cells. `2n` works only for the iterative layout.
- **Inconsistent ranges.** The iterative query above is half-open `[l, r)`; the recursive one is inclusive `[l, r]`. Convert 1-indexed input carefully (`query(x-1, y)` versus `query(1, 0, n-1, x-1, y-1)`).
- **Wrong identity.** Returning 0 for a disjoint node in a min-tree silently gives 0 as the answer.
- **Non-commutative combine in the iterative tree** with a single accumulator, or with non-power-of-two n.
- **Overflow.** Sums of 2·10^5 values up to 10^9 need `long long`.
- **Forgetting to pull up** (`t[x] = combine(children)`) after an update.
- **O(log² n) where O(log n) is needed.** Binary search + range query usually passes for 2·10^5, but a walk is both faster and simpler once learned.
- **Recursing into a node whose range is empty** when building with n = 0; guard against empty input if the problem allows it.

10. Practice problems

Warm-up:

- CSES 1648 — *Dynamic Range Sum Queries* — https://cses.fi/problemset/task/1648
- CSES 1649 — *Dynamic Range Minimum Queries* — https://cses.fi/problemset/task/1649
- CSES 1650 — *Range Xor Queries* — https://cses.fi/problemset/task/1650

Core:

- CSES 1143 — *Hotel Queries* — https://cses.fi/problemset/task/1143
- CSES 1749 — *List Removals* — https://cses.fi/problemset/task/1749
- CSES 1144 — *Salary Queries* — https://cses.fi/problemset/task/1144
- CSES 1190 — *Subarray Sum Queries* — https://cses.fi/problemset/task/1190
- Codeforces 339D — *Xenia and Bit Operations* — https://codeforces.com/problemset/problem/339/D

Challenge:

- CSES 2166 — *Prefix Sum Queries* — https://cses.fi/problemset/task/2166
- CSES 2206 — *Pizzeria Queries* — https://cses.fi/problemset/task/2206
- Codeforces 474F — *Ant colony* — https://codeforces.com/problemset/problem/474/F

- Codeforces 380C — *Sereja and Brackets* — https://codeforces.com/problemset/problem/380/C
- Codeforces EDU "Segment Tree, part 1" (steps 1–4) — https://codeforces.com/edu/course/2/lesson/4

11. Further reading

- CP-Algorithms, "Segment Tree" — https://cp-algorithms.com/data_structures/segment_tree.html
- Al.Cash, "Efficient and easy segment trees" (the bottom-up version) — https://codeforces.com/blog/entry/18051
- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 9 "Range queries".
- USACO Guide, "Point Update Range Sum" — https://usaco.guide/gold/PURS
- Next lectures: Week 24 (Fenwick trees and sparse tables — lighter alternatives for special cases) and Week 25 (lazy propagation for range updates).